BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM
KHOA ĐIỆN - ĐIỆN TỬ



BÁO CÁO CUỐI KỲ

MÔN HỌC: CẤU TRÚC RỜI RẠC

DỰ ĐOÁN THỜI GIAN ÙN TẮC GIAO THÔNG

Giảng viên hướng dẫn: TS. Phan Học
Mã lớp học phần: [UTE] 261DSCC235864_03

Danh sách sinh viên thực hiện:
1. Nguyễn Lê Phúc Thịnh 25139047
2. Nguyễn Tuấn Thịnh 25139048

TP. Hồ Chí Minh, tháng 10 năm 2026

Dự đoán thời gian ùn tắc giao thông

Nguyễn Lê Phúc Thịnh, Nguyễn Tuấn Thịnh

Khoa Điện - Điện tử, Trường Đại học Công nghệ Kỹ thuật TP.HCM

Giảng viên hướng dẫn: TS. Phan Học

Tóm tắt nội dung

... Ghi gì đó ở đây ...

1 Giới thiệu

Nút giao Ngã tư Hàng Xanh là một trong những nút giao thông trọng điểm và phức tạp tại khu vực TP.HCM, nơi tập trung lưu lượng phương tiện khổng lồ với đa dạng loại xe (xe máy, ô tô, xe tải, container). Do sự biến động liên tục của dòng xe theo từng khung giờ trong ngày và các ngày trong tuần, khu vực này thường xuyên rơi vào tình trạng ùn tắc cục bộ, gây ách tắc dây chuyền và ảnh hưởng lớn đến việc lưu thông. Do đó, bài toán cần đặt ra là cần xây dựng một hệ thống giám sát tự động có khả năng thu thập dữ liệu từ camera thời gian thực để mô hình hóa trạng thái giao thông, từ đó đưa ra dự đoán sớm về thời điểm ùn tắc.

2 Cơ sở lý thuyết

2.1 Hàm số (Functions)

Theo định nghĩa trong giáo trình (Rosen, 8th Ed., Chương 2), hàm số $f : A \to B$ được sử dụng để thiết lập một phép gán mỗi phần tử của tập hợp A với đúng một phần tử của tập hợp B. Trong bài toán này, A là không gian trạng thái đầu vào chứa các vector dữ liệu lưu lượng đếm được từ khung hình camera (số xe máy, ô tô, xe tải, xe buýt), và B là tập hợp các mức độ ùn tắc hoặc thời gian chờ dự kiến. Quá trình tiền xử lý dữ liệu còn áp dụng các hàm sàn (floor function) và hàm trần (ceiling function) để rời rạc hóa các giá trị thời gian liên tục thành các khoảng (time bins) cố định. Điều này giúp thuật toán dễ dàng phân loại và ánh xạ trạng thái giao thông với chi phí tính toán (Computational Cost) thấp nhất.

2.2 Hệ thức truy hồi (Recursion)

Lưu lượng giao thông hoạt động như một hệ nhân quả theo thời gian rời rạc, nghĩa là trạng thái ùn tắc tại thời điểm hiện tại chịu ảnh hưởng trực tiếp từ lưu lượng xe trong các mốc thời gian quá khứ. Dựa trên lý thuyết về quan hệ truy hồi (Rosen, 8th Ed., Chương 8), mật độ phương tiện tại thời điểm t, ký hiệu là N_t, được mô hình hóa dưới dạng một hệ thức truy hồi tuyến tính:

$$N_t = c_1 N_{t-1} + c_2 N_{t-2} + \ldots + c_k N_{t-k} + F(t)$$

Trong đó, c_i là các hằng số trọng số thể hiện mức độ ảnh hưởng của quá khứ, và $F(t)$ là hàm đánh giá tác động ngoại cảnh. Việc biểu diễn tính nhân quả thông qua hệ thức truy hồi cho phép phần mềm dự báo được lưu lượng luồng xe trong tương lai gần chỉ bằng cách truy xuất một vài trạng thái bộ nhớ gần nhất, giúp tối ưu hóa được độ phức tạp của thuật toán.

2.3 Đồ thị (Graphs) & Quan hệ (Relations)

Không gian giao thông tại nút giao Hàng Xanh trong khung hình camera được trừu tượng hóa thành một đồ thị có hướng và có trọng số $G = (V, E)$ (Rosen, 8th Ed., Chương 10). Tập đỉnh V đại diện cho các làn xe hoặc khu vực chờ, tập cạnh E đại diện cho hướng đi của luồng phương tiện, và trọng số của cạnh chính là số lượng xe hiện hành được trích xuất từ camera. Sự tương tác giữa các luồng xe được biểu diễn bằng một ma trận kề (Adjacency Matrix) đại diện cho một Quan hệ (Relation) R trên tập V (Rosen, 8th Ed., Chương 9). Chẳng hạn, nếu luồng xe đi thẳng từ làn v_i gây xung đột và cản trở luồng xe rẽ trái từ làn v_j, hệ thống sẽ ghi nhận $(v_i, v_j) \in R$. Phân tích các ma trận quan hệ và bậc của các đỉnh (degree of vertices) trên đồ thị giúp thuật toán khoanh vùng chính xác các điểm nghẽn (bottlenecks) ngay bên trong giao lộ để phát ra cảnh báo kẹt xe.

3 Data Collection & Implementation

Test....

4 Kết luận

...

A Mã nguồn (Source Codes)

A.1 Các thư viện được sử dụng

```
import requests
import cv2
import numpy as np
import time

```

A.2 Get_frame.py

```
def get_traffic_image(name="Camera 1"):
    global cam_id
    timestamp = int(time.time() * 1000)
    url =
     ↪  f"https://giaothong.hochiminhcity.gov.vn/render/ImageHandler.ashx?id={cam_id[name]}&t={tim

    headers = {
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36",
        "Referer": "https://giaothong.hochiminhcity.gov.vn/Map.aspx"
    }

    try:
        response = requests.get(url, headers=headers, timeout=5)
        if response.status_code == 200:
            img_array = np.asarray(bytearray(response.content), dtype=np.uint8)
            frame = cv2.imdecode(img_array, cv2.IMREAD_COLOR)
            return frame
        else:
            print(f"Server response error: {response.status_code}")
    except requests.exceptions.RequestException as e:
        print(f"Network error: {e}")

    return None
```

A.3 Yolo_detect.py

```
def callback(image_slice: np.ndarray) -> sv.Detections:
    result = model(image_slice)[0]
    return sv.Detections.from_ultralytics(result)

def image_processing(frame):
    slicer = sv.InferenceSlicer(callback=callback, slice_wh=(256, 256))
    detections = slicer(frame)

    # Đếm theo class ID của YOLO
    class_ids = detections.class_id

    person_count = np.sum(class_ids == 0)
    car_count = np.sum(class_ids == 2)
    motorbike_count = np.sum(class_ids == 3)
    bus_count = np.sum(class_ids == 5)
```

```
    truck_count = np.sum(class_ids == 7)
    total_vehicles = car_count + motorbike_count + bus_count + truck_count
    total_objects = len(class_ids)

    print(f"Person: {person_count}")
    print(f"Car: {car_count}")
    print(f"Motorbike: {motorbike_count}")
    print(f"Bus: {bus_count}")
    print(f"Truck: {truck_count}")
    print(f"Total vehicles: {total_vehicles}")
    print(f"Total objects detected: {total_objects}")

    # Weighted Traffic Impact (WTI)
    car_w = 4
    motorbike_w = 1
    bus_w = 7
    truck_w = 6

    weighted_traffic_impact = (
            int(car_count) * car_w
            + int(motorbike_count) * motorbike_w
            + int(bus_count) * bus_w
            + int(truck_count) * truck_w
    )

    weighted_traffic_impact_norm = (
        weighted_traffic_impact / total_vehicles if total_vehicles > 0 else 0.0
    )

    box_annotator = sv.BoxAnnotator(thickness=1)

    annotated_frame = box_annotator.annotate(
        scene=frame.copy(),
        detections=detections
    )

    current_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    cv2.putText(annotated_frame, f"{current_time}", (15, 35),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 255), 2)

    return annotated_frame, person_count, car_count, motorbike_count, bus_count,
     ↪  truck_count, total_vehicles, weighted_traffic_impact, weighted_traffic_impact_norm
```